

□ Pytest —Atelier Complet

Le support est propriété de **Bechir Bejaoui**

1 □ Pourquoi tester son code ?

Les tests permettent de :

- Détecter les régressions avant la production
- Documenter le comportement attendu
- Refactoriser sans crainte
- Travailler en équipe sereinement
- Automatiser la validation continue (CI/CD)

Exemple réel :

Un calcul de facture, une règle métier RH, un algorithme de scoring...
Tout code critique doit être couvert par des tests.

2 □ Installation et premier test

Installation

```
pip install pytest
```

Arborescence de référence

```
projet/  
├── calcul.py  
└── test_calcul.py
```

Exemple minimal

calcul.py

```
def addition(a: float, b: float) -> float:  
    """Retourne la somme de deux nombres."""  
    return a + b
```

test_calcul.py

```

from calcul import addition

def test_addition_entiers():
    # Arrange
    a, b = 3, 4

    # Act
    resultat = addition(a, b)

    # Assert
    assert resultat == 7

```

Lancer les tests

```

pytest                    # tous les tests
pytest test_calcul.py    # fichier précis
pytest -v                # mode verbeux

```

3 Conventions de nommage

pytest découvre automatiquement les tests selon ces règles :

- Fichiers commençant par test_ ou finissant par _test.py
- Fonctions commençant par test_
- Classes commençant par Test (sans __init__)

```

# ? Découvert automatiquement
def test_addition():
    ...

class TestCalcul:
    def test_soustraction(self):
        ...

# ? Ignoré par pytest
def verifier_addition():
    ...

```

4 assert —les assertions

pytest réécrit les assertions pour afficher des messages clairs en cas d'échec.

```
def test_assertions_variees():
    # Égalité
    assert 2 + 2 == 4

    # Inégalité
    assert 10 != 0

    # Appartenance
    assert "admin" in ["admin", "user"]

    # Type
    assert isinstance(3.14, float)

    # Valeur booléenne
    assert len([1, 2, 3]) > 0
```

En cas d'échec, pytest affiche :

AssertionError: assert 5 == 4

5 □ `pytest.raises` — tester les exceptions

```
projet/
├── validation.py
└── test_validation.py
```

`validation.py`

```
def diviser(a: float, b: float) -> float:
    """Divise a par b. Lève ValueError si b est nul."""
    if b == 0:
        raise ValueError("Division par zéro interdite")
    return a / b
```

`test_validation.py`

```
import pytest
from validation import diviser

def test_division_normale():
    assert diviser(10, 2) == 5.0

def test_division_par_zero():
    # Vérifier qu'une exception est bien levée
    with pytest.raises(ValueError) as exc_info:
        diviser(10, 0)
```

```
# Vérifier le message de l'exception
assert "zéro" in str(exc_info.value)
```

6 □ Fixtures —préparer le contexte

Une fixture prépare les données ou ressources nécessaires à un test.

```
projet/
├─ panier.py
└─ test_panier.py
```

panier.py

```
class Panier:
    def __init__(self):
        self.articles = []

    def ajouter(self, nom: str, prix: float):
        self.articles.append({"nom": nom, "prix": prix})

    def total(self) -> float:
        return sum(a["prix"] for a in self.articles)
```

test_panier.py

```
import pytest
from panier import Panier

@pytest.fixture
def panier_rempli():
    """Fixture : panier avec deux articles."""
    p = Panier()
    p.ajouter("Clavier", 49.99)
    p.ajouter("Souris", 29.99)
    return p

def test_total(panier_rempli):
    assert panier_rempli.total() == pytest.approx(79.98)

def test_nombre_articles(panier_rempli):
    assert len(panier_rempli.articles) == 2
```

La fixture est injectée automatiquement par son nom.

7 `pytest.approx` —comparer des flottants

Comparer des flottants directement est dangereux :

```
# ❗ Peut échouer à cause de la précision flottante
assert 0.1 + 0.2 == 0.3
```

```
# ✅ Correct
import pytest
assert 0.1 + 0.2 == pytest.approx(0.3)

def test_tva():
    prix_ht = 100.0
    tva = 0.20
    prix_ttc = prix_ht * (1 + tva)

    assert prix_ttc == pytest.approx(120.0)
```

8 `Parametrize` —tester plusieurs cas

`@pytest.mark.parametrize` évite la duplication de tests similaires.

```
projet/
├── calcul.py
└── test_calcul.py
```

calcul.py

```
def est_pair(n: int) -> bool:
    """Retourne True si n est pair."""
    return n % 2 == 0
```

test_calcul.py

```
import pytest
from calcul import est_pair

@pytest.mark.parametrize("nombre, attendu", [
    (2, True),
    (3, False),
    (0, True),
    (-4, True),
    (7, False),
])
def test_est_pair(nombre, attendu):
    assert est_pair(nombre) == attendu
```

Un seul test couvre 5 cas —chaque ligne est une exécution indépendante.

9 □ conftest.py —fixtures partagées

conftest.py centralise les fixtures accessibles à tout le projet.

```
projet/
├── conftest.py
├── test_panier.py
└── test_commande.py
```

conftest.py

```
import pytest
from panier import Panier

@pytest.fixture
def panier_vide():
    """Fixture disponible dans tous les fichiers de test."""
    return Panier()

@pytest.fixture
def panier_rempli():
    p = Panier()
    p.ajouter("Clavier", 49.99)
    p.ajouter("Souris", 29.99)
    return p
```

test_panier.py

```
def test_panier_vide(panier_vide):
    assert panier_vide.total() == 0

def test_panier_rempli(panier_rempli):
    assert len(panier_rempli.articles) == 2
```

Pas besoin d'importer les fixtures —pytest les injecte automatiquement.

□ Portée des fixtures —scope

```
import pytest

@pytest.fixture(scope="function")  # défaut : recréé à chaque test
def connexion_legere():
```

```

        return {"host": "localhost"}

@pytest.fixture(scope="module")    # une fois par fichier
def connexion_module():
    print("\n[Connexion ouverte]")
    yield {"host": "db.prod"}
    print("\n[Connexion fermée]")

@pytest.fixture(scope="session")    # une fois pour toute la session
def config_globale():
    return {"env": "test", "debug": True}

```

Scope	Durée de vie
function	Recréé pour chaque test (défaut)
class	Partagé dans une classe de tests
module	Partagé dans un fichier
session	Partagé sur toute la session

1 □1 □ Mocking —isoler les dépendances

unittest.mock permet de remplacer des dépendances externes (API, base de données...).

```

projet/
├── service.py
└── test_service.py

```

service.py

```

import requests

def get_statut_serveur(url: str) -> str:
    """Appelle une API et retourne le statut."""
    reponse = requests.get(url)
    if reponse.status_code == 200:
        return "operationnel"
    return "hors service"

```

test_service.py

```

from unittest.mock import patch, MagicMock
from service import get_statut_serveur

def test_serveur_operationnel():

```

```

# Remplacer requests.get par un faux objet
with patch("service.requests.get") as mock_get:
    mock_get.return_value = MagicMock(status_code=200)

    resultat = get_statut_serveur("http://fakeurl.com")

    assert resultat == "operationnel"
    mock_get.assert_called_once_with("http://fakeurl.com")

def test_serveur_hors_service():
    with patch("service.requests.get") as mock_get:
        mock_get.return_value = MagicMock(status_code=500)

        resultat = get_statut_serveur("http://fakeurl.com")

        assert resultat == "hors service"

```

1 □ 2 □ Marks —catégoriser les tests

```

import pytest

@pytest.mark.smoke
def test_ping():
    assert True

@pytest.mark.slow
def test_traitement_lourd():
    import time
    time.sleep(0.1)
    assert True

@pytest.mark.skip(reason="Fonctionnalité non implémentée")
def test_futur():
    assert False

@pytest.mark.xfail(reason="Bug connu #42")
def test_comportement_bugge():
    assert 1 == 2

pytest -m smoke           # uniquement les tests smoke
pytest -m "not slow"      # exclure les tests lents
pytest -v --tb=short      # traceback court

```

Déclarer les marks dans `pytest.ini` :

```
[pytest]
markers =
    smoke: tests de fumée rapides
    slow: tests lents
```

1 □3 □ Structure AAA —Arrange / Act / Assert

Le pattern AAA structure chaque test de façon lisible.

```
from panier import Panier

def test_ajout_article():
    # Arrange — préparer le contexte
    panier = Panier()

    # Act — exécuter l'action testée
    panier.ajouter("Écran", 249.99)

    # Assert — vérifier le résultat
    assert len(panier.articles) == 1
    assert panier.total() == pytest.approx(249.99)
```

Règle : **un test = une responsabilité = une assertion principale.**

1 □4 □ Fixture avec yield —setup et teardown

yield sépare la préparation (setup) du nettoyage (teardown).

```
import pytest

@pytest.fixture
def fichier_temporaire(tmp_path):
    """Crée un fichier de test, le supprime après."""
    fichier = tmp_path / "data.txt"
    fichier.write_text("contenu initial")

    yield fichier    # Le test s'exécute ici

    # Teardown automatique après le test
    if fichier.exists():
        fichier.unlink()
```

```
def test_lecture_fichier(fichier_temporaire):
    contenu = fichier_temporaire.read_text()
    assert contenu == "contenu initial"
```

tmp_path est une fixture native pytest qui crée un dossier temporaire.

1 5 5 Tester une classe complète

```
projet/
├── compte.py
└── test_compte.py
```

compte.py

```
class CompteBancaire:
    def __init__(self, titulaire: str, solde: float = 0):
        self.titulaire = titulaire
        self.__solde = solde

    def deposter(self, montant: float):
        if montant <= 0:
            raise ValueError("Montant invalide")
        self.__solde += montant

    def retirer(self, montant: float):
        if montant > self.__solde:
            raise ValueError("Solde insuffisant")
        self.__solde -= montant

    def solde(self) -> float:
        return self.__solde
```

test_compte.py

```
import pytest
from compte import CompteBancaire

@pytest.fixture
def compte():
    return CompteBancaire("Bechir", solde=500.0)

def test_depot(compte):
    compte.deposer(200)
    assert compte.solde() == 700.0

def test_retrait(compte):
```

```

    compte.retirer(100)
    assert compte.solde() == 400.0

def test_depot_invalide(compte):
    with pytest.raises(ValueError, match="Montant invalide"):
        compte.deposer(-50)

def test_retrait_insuffisant(compte):
    with pytest.raises(ValueError, match="Solde insuffisant"):
        compte.retirer(1000)

```

1 □6 □ Coverage —mesurer la couverture

pip install pytest-cov

```

pytest --cov=calcul # couverture du module calcul
pytest --cov=calcul --cov-report=term-missing # lignes non couvertes
pytest --cov=. --cov-report=html # rapport HTML dans htmlcov/

```

Exemple de rapport terminal :

Name	Stmts	Miss	Cover
calcul.py	12	2	83%
panier.py	8	0	100%
TOTAL	20	2	90%

Objectif recommandé en production : **80% minimum**.

1 □7 □ Organisation d'un projet réel

```

projet/
├── src/
│   ├── stockpro/
│   │   ├── __init__.py
│   │   ├── calcul.py
│   │   └── panier.py
├── tests/
│   ├── conftest.py
│   ├── test_calcul.py
│   └── test_panier.py
└── pytest.ini

```

└─ requirements.txt

pytest.ini

```
[pytest]
testpaths = tests
addopts = -v --tb=short
markers =
    smoke: tests rapides de vérification
    integration: tests d'intégration
```

requirements.txt

```
pytest
pytest-cov
```

1 8 Bonnes pratiques

- Un test = une responsabilité unique
 - Nommer les tests de façon explicite : test_retrait_solde_insuffisant
 - Utiliser des fixtures plutôt que répéter le setup
 - Centraliser les fixtures partagées dans conftest.py
 - Tester les cas nominaux ET les cas d'erreur
 - Ne jamais tester l'implémentation —tester le comportement
 - Viser 80% de couverture minimum
 - Exécuter les tests avant chaque commit
-

1 9 pytest en CI/CD

```
# .github/workflows/tests.yml
name: Tests

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-python@v4
        with:
          python-version: "3.11"
```

- run: `pip install -r requirements.txt`
- run: `pytest --cov=src --cov-report=term-missing`

Les tests s'exécutent automatiquement à chaque push.

2 00 Synthèse

- `pytest` découvre automatiquement les fichiers `test_*.py`
 - `assert` est réécrit par `pytest` pour des messages clairs
 - `pytest.raises` vérifie qu'une exception est bien levée
 - `@pytest.fixture` prépare et nettoie le contexte de test
 - `conftest.py` centralise les fixtures partagées
 - `@pytest.mark.parametrize` couvre plusieurs cas en un test
 - `unittest.mock.patch` isole les dépendances externes
 - `pytest --cov` mesure la couverture du code
-

Questions entretiens

Voici **20 questions niveau intermédiaire** sous forme **Énoncé + Solution**, avec à chaque fois :

- une **explication en haut du script**
 - un **enchaînement clair** (démonstration → affichage → conclusion)
 - du **code commenté**
-

1) Pourquoi `assert a == b` dans `pytest` est plus puissant qu'en Python pur ?

Énoncé

Que se passe-t-il quand un `assert` échoue dans `pytest`, et pourquoi c'est différent du comportement Python standard ?

Solution

```
"""
Explication
- Python standard: assert lève AssertionError avec peu d'information.
- pytest réécrit (introspect) l'assert pour afficher les valeurs réelles.
- C'est le mécanisme d'assertion rewriting de pytest.
"""

# test_demo.py
def addition(a, b):
    return a + b

def test_echec_informatif():
    resultat = addition(2, 3)
    attendu = 10

    # En cas d'échec, pytest affiche :
    # AssertionError: assert 5 == 10
    # + where 5 = addition(2, 3)
    assert resultat == attendu
```

2) Quelle différence entre une fixture scope="function" et scope="module" ?

Énoncé

Pourquoi le scope d'une fixture impacte les performances et l'isolation des tests ?

Solution

```
"""
Explication
- scope="function" : fixture recréée pour chaque test → isolation maximale.
- scope="module" : fixture créée une fois pour tous les tests du fichier.
- Utiliser module/session pour les ressources coûteuses (connexion DB, serveur).
"""

import pytest

@pytest.fixture(scope="function")
def panier_isole():
```

```

# Recréé à chaque test → aucune pollution entre tests
from panier import Panier
return Panier()

@pytest.fixture(scope="module")
def connexion_db():
    # Crée une seule fois pour le fichier → plus rapide
    print("\n[DB connectée]")
    yield {"host": "localhost", "db": "test"}
    print("\n[DB déconnectée]")

def test_panier_vide(panier_isole):
    assert panier_isole.total() == 0

def test_panier_apres(panier_isole):
    # Nouveau panier — pas de pollution du test précédent
    assert len(panier_isole.articles) == 0

```

3) Comment tester qu'une exception contient un message précis ?

Énoncé

pytest.raises capture l'exception, mais comment vérifier le message exactement ?

Solution

```

"""
Explication
- pytest.raises() retourne un ExceptionInfo.
- exc_info.value contient l'exception elle-même.
- match= permet une vérification par expression régulière directement.
"""

import pytest

def retirer(solde, montant):
    if montant > solde:
        raise ValueError(f"Solde insuffisant : {solde} disponible")
    return solde - montant

def test_message_exception_via_match():

```

```

# match= vérifie le message avec une regex
with pytest.raises(ValueError, match="Solde insuffisant"):
    retirer(100, 500)

def test_message_exception_via_value():
    with pytest.raises(ValueError) as exc_info:
        retirer(100, 500)

    # Accès complet au message
    assert "100" in str(exc_info.value)

```

4) Pourquoi ne pas mettre de logique dans une fixture ?

Énoncé

Quel problème survient quand une fixture contient trop de logique métier ?

Solution

```

"""
Explication
- Une fixture doit préparer un contexte, pas tester une logique.
- Si la fixture échoue, le test est marqué ERROR, pas FAILED.
- La logique dans la fixture rend les tests difficiles à déboguer.
"""

```

```

import pytest
from panier import Panier

# ☹️ Mauvaise pratique : Logique dans la fixture
@pytest.fixture
def panier_avec_remise():
    p = Panier()
    p.ajouter("Clavier", 100.0)
    total = p.total()
    remise = total * 0.10 # Logique métier ici → à éviter
    return p, remise

# ☺️ Bonne pratique : fixture simple, logique dans le test
@pytest.fixture
def panier_simple():
    p = Panier()

```

```

    p.ajouter("Clavier", 100.0)
    return p

def test_remise(panier_simple):
    # La logique métier est dans le test, pas dans la fixture
    remise = panier_simple.total() * 0.10
    assert remise == pytest.approx(10.0)

```

5) Comment tester plusieurs cas sans dupliquer le code ?

Énoncé

Comment parametrize évite la répétition tout en gardant chaque cas indépendant ?

Solution

```

"""
Explication
- @pytest.mark.parametrize génère un test par ligne de paramètres.
- Chaque cas est indépendant : un échec n'affecte pas les autres.
- L'ID affiché est généré automatiquement ou peut être personnalisé.
"""

import pytest

def calculer_ttc(prix_ht: float, taux: float = 0.20) -> float:
    return round(prix_ht * (1 + taux), 2)

@pytest.mark.parametrize("prix_ht, taux, attendu", [
    (100.0, 0.20, 120.0),
    (50.0, 0.10, 55.0),
    (0.0, 0.20, 0.0),
    (200.0, 0.00, 200.0),
], ids=["tva_normale", "tva_reduite", "prix_nul", "sans_tva"])
def test_calcul_ttc(prix_ht, taux, attendu):
    assert calculer_ttc(prix_ht, taux) == pytest.approx(attendu)

```

6) Pourquoi patch doit cibler le bon emplacement ?

Énoncé

Pourquoi `patch("requests.get")` peut ne pas fonctionner alors que `patch("service.requests.get")` fonctionne ?

Solution

```
"""
Explication
- patch remplace l'objet là où il est utilisé, pas là où il est défini.
- Si service.py importe requests, il faut patcher service.requests.get.
- Patcher requests.get directement ne touche pas la référence dans service.py.
"""

import pytest
from unittest.mock import patch, MagicMock

# service.py contient : import requests / requests.get(url)

def test_patch_correct():
    # Patch au bon endroit : là où requests est utilisé
    with patch("service.requests.get") as mock_get:
        mock_get.return_value = MagicMock(status_code=200)
        from service import get_statut_serveur
        assert get_statut_serveur("http://test.com") == "operationnel"

def test_patch_incorrect():
    # Patch au mauvais endroit : n'affecte pas service.py
    with patch("requests.get") as mock_get:
        mock_get.return_value = MagicMock(status_code=200)
        # service.py utilise sa propre référence à requests
        # ce patch peut ne pas fonctionner selon l'ordre d'import
        pass
```

7) Que vérifie `mock.assert_called_once_with()` ?

Énoncé

Comment s'assurer qu'une dépendance mockée a été appelée exactement comme prévu ?

Solution

```
"""
Explication
- assert_called_once_with() vérifie que le mock a été appelé UNE SEULE fois
  avec les arguments exacts.
- assert_called_with() vérifie uniquement le dernier appel.
- call_count donne le nombre total d'appels.
"""

from unittest.mock import patch, MagicMock
from service import get_statut_serveur

def test_appel_api_verifie():
    with patch("service.requests.get") as mock_get:
        mock_get.return_value = MagicMock(status_code=200)

        url = "http://api.example.com/health"
        get_statut_serveur(url)

        # Vérifier que requests.get a été appelé une fois avec la bonne URL
        mock_get.assert_called_once_with(url)

        # Vérifier le nombre d'appels
        assert mock_get.call_count == 1
```

8) Différence entre tmp_path et monkeypatch ?

Énoncé

Quand utiliser la fixture tmp_path et quand utiliser monkeypatch ?

Solution

```
"""
Explication
- tmp_path : fixture native pytest pour créer des fichiers/dossiers temporaires.
  Nettoyage automatique après le test.
- monkeypatch : modifie temporairement des attributs, variables d'environnement,
  ou fonctions. Restauration automatique après le test.
"""

import os
```

```

def test_fichier_temporaire(tmp_path):
    # tmp_path est un dossier unique par test
    fichier = tmp_path / "rapport.txt"
    fichier.write_text("données de test")

    assert fichier.exists()
    assert fichier.read_text() == "données de test"
    # Supprimé automatiquement après le test

def lire_config():
    return os.getenv("APP_ENV", "production")

def test_variable_environnement(monkeypatch):
    # Modifier temporairement une variable d'environnement
    monkeypatch.setenv("APP_ENV", "test")

    assert lire_config() == "test"
    # APP_ENV est restauré à sa valeur d'origine après le test

```

9) Comment tester une méthode privée ?

Énoncé

Faut-il tester les méthodes privées directement ? Quelle est la bonne approche ?

Solution

```

"""
Explication
- Les méthodes privées sont un détail d'implémentation.
- Tester via l'interface publique est la bonne pratique.
- Si une méthode privée est trop complexe à tester indirectement,
  c'est souvent le signe qu'elle devrait être extraite dans une classe séparée.
"""

```

```

class CalculeurFacture:
    def __calculer_remise(self, montant: float, taux: float) -> float:
        # Logique privée
        return montant * taux

    def total_apres_remise(self, montant: float, taux: float) -> float:
        remise = self.__calculer_remise(montant, taux)

```

```

        return montant - remise

# 📄 Tester via L'interface publique
def test_total_apres_remise():
    calc = CalculateurFacture()
    # On teste Le comportement observable, pas L'implémentation
    assert calc.total_apres_remise(100.0, 0.10) == pytest.approx(90.0)
    assert calc.total_apres_remise(200.0, 0.20) == pytest.approx(160.0)

```

10) Pourquoi xfail est préférable à skip pour un bug connu ?

Énoncé

Quelle différence entre skip et xfail dans la gestion des tests problématiques ?

Solution

```

"""
Explication
- skip : Le test ne s'exécute pas du tout → masque Le problème.
- xfail : Le test s'exécute et on attend un échec → documente Le bug.
- Si un test xfail passe de façon inattendue → XPASS → signal important.
"""

```

```

import pytest

@pytest.mark.skip(reason="Pas encore implémenté")
def test_fonctionnalite_future():
    # Ne s'exécute jamais → on ne sait pas si ça marche
    assert False

@pytest.mark.xfail(reason="Bug connu #42 — division retourne None")
def test_bug_connu():
    # S'exécute → documente que Le bug existe
    # Si Le bug est corrigé → XPASS → alerte L'équipe
    from calcul import diviser
    assert diviser(10, 2) == 5

```

11) Comment tester du code asynchrone ?

Énoncé

Comment pytest gère-t-il les fonctions async ?

Solution

```
"""
Explication
- pytest ne gère pas nativement l'async.
- pytest-asyncio ajoute ce support.
- Décorer le test avec @pytest.mark.asyncio.
"""

# pip install pytest-asyncio

import pytest
import asyncio

async def fetch_donnees(url: str) -> dict:
    """Simule une requête asynchrone."""
    await asyncio.sleep(0.01)
    return {"url": url, "statut": "ok"}

@pytest.mark.asyncio
async def test_fetch_donnees():
    resultat = await fetch_donnees("http://api.test.com")

    assert resultat["statut"] == "ok"
    assert "url" in resultat
```

12) Pourquoi un test lent peut masquer un problème de design ?

Énoncé

Un test qui prend plusieurs secondes est souvent le symptôme d'un problème architectural.

Solution

```
"""
```

Explication

- Un test lent indique souvent une dépendance réelle (base de données, réseau, fichier)
- La solution : isoler via mock ou fixture légère.
- Règle : un test unitaire doit s'exécuter en moins de 100ms.

```
"""
```

```
import pytest
from unittest.mock import patch, MagicMock

# 🐞 Test Lent : appel réseau réel
def test_api_lent():
    import requests
    # response = requests.get("http://api.externe.com") # 2-5 secondes
    pass

# 🐞 Test rapide : dépendance mockée
def test_api_rapide():
    with patch("service.requests.get") as mock_get:
        mock_get.return_value = MagicMock(status_code=200)
        from service import get_statut_serveur

        resultat = get_statut_serveur("http://api.externe.com")

        # Exécution en microsecondes
        assert resultat == "operationnel"
```

13) Comment tester les interactions entre objets ?

Énoncé

Comment vérifier qu'un objet appelle correctement ses collaborateurs ?

Solution

```
"""
```

Explication

- Un test d'interaction vérifie les appels entre objets, pas seulement le résultat.
- MagicMock simule un collaborateur et enregistre tous les appels.

```
"""
```

```
from unittest.mock import MagicMock
```

```

class ServiceNotification:
    def envoyer(self, message: str, destinataire: str):
        pass # Envoie un email réel

class ServiceCommande:
    def __init__(self, notifieur: ServiceNotification):
        self.notifieur = notifieur

    def passer_commande(self, produit: str, client: str):
        # Logique métier
        confirmation = f"Commande {produit} confirmée"
        # Notifier le client
        self.notifieur.envoyer(confirmation, client)
        return confirmation

def test_notification_envoyee():
    # Arrange
    mock_notif = MagicMock(spec=ServiceNotification)
    service = ServiceCommande(notifieur=mock_notif)

    # Act
    service.passer_commande("Clavier", "bechir@example.com")

    # Assert — vérifier que la notification a bien été envoyée
    mock_notif.envoyer.assert_called_once_with(
        "Commande Clavier confirmée",
        "bechir@example.com"
    )

```

14) Pourquoi conftest.py est préférable aux imports de fixtures ?

Énoncé

Peut-on importer une fixture depuis un autre fichier ? Pourquoi conftest.py est recommandé ?

Solution

"""

Explication

- pytest ne permet pas d'importer des fixtures comme des fonctions ordinaires.

- *conftest.py* est automatiquement découvert par pytest dans le dossier et ses parents.
- C'est le seul mécanisme officiel pour partager des fixtures entre fichiers.

```

# conftest.py — à la racine du dossier tests/
import pytest
from compte import CompteBancaire

@pytest.fixture
def compte_test():
    """Disponible dans tous les fichiers du dossier tests/ et sous-dossiers."""
    return CompteBancaire("Bechir", solde=1000.0)

# test_compte.py — utilise la fixture sans l'importer
def test_solde_initial(compte_test):
    assert compte_test.solde() == 1000.0

# test_operations.py — même fixture disponible
def test_depot(compte_test):
    compte_test.deposer(500)
    assert compte_test.solde() == 1500.0

```

15) Comment mesurer la couverture d'un module précis ?

Énoncé

Comment cibler la couverture sur votre code source et ignorer les dépendances ?

Solution

```

"""
Explication
- --cov=src mesure uniquement le dossier src/.
- --cov-report=term-missing affiche les numéros de lignes non couvertes.
- .coveragerc configure les exclusions (fichiers de config, migrations...).
"""

# Commandes utiles
# pytest --cov=src --cov-report=term-missing
# pytest --cov=src --cov-report=html           → rapport HTML dans htmlcov/
# pytest --cov=src --cov-fail-under=80        → échec si couverture < 80%

```

```
# .coveragerc
"""
[run]
source = src
omit =
    */migrations/*
    */config.py
    */__init__.py

[report]
show_missing = True
fail_under = 80
"""
```

```
# Exemple de sortie terminal
"""
```

Name	Stmts	Miss	Cover	Missing
src/calcul.py	15	2	87%	24, 31
src/panier.py	8	0	100%	
src/compte.py	20	3	85%	45-47
TOTAL	43	5	88%	

```
"""
```

16) Différence entre test unitaire, test d'intégration et test end-to-end ?

Énoncé

Comment organiser et distinguer les différents niveaux de tests dans un projet ?

Solution

```
"""
Explication
- Unitaire : teste une fonction/classe isolée (mock des dépendances).
- Intégration : teste plusieurs composants ensemble (base de données réelle).
- End-to-end : teste le flux complet (interface + API + base de données).
"""
```

```
import pytest
```

```
# — Test unitaire — isolé, rapide —————  
def test_calcul_ttc_unitaire():  
    from calcul import calculer_ttc  
    # Aucune dépendance externe — sous la milliseconde  
    assert calculer_ttc(100.0, 0.20) == pytest.approx(120.0)
```

```
# — Test d'intégration — composants réels —————  
@pytest.mark.integration  
def test_panier_avec_compte(panier_rempli, compte_test):  
    # Deux composants réels interagissent  
    montant = panier_rempli.total()  
    compte_test.retirer(montant)  
    assert compte_test.solde() < 1000.0
```

```
# — Organisation recommandée —————  
"""
```

```
tests/  
├── unit/  
│   ├── test_calcul.py  
│   └── test_panier.py  
├── integration/  
│   └── test_panier_compte.py  
└── e2e/  
    └── test_flux_achat.py  
"""
```

17) Pourquoi tester les cas limites (edge cases) ?

Énoncé

Quels cas limites tester systématiquement ?

Solution

```
"""  
Explication  
- Les bugs se cachent aux frontières : 0, négatif, liste vide, None, chaîne vide.  
- Les cas nominaux seuls ne suffisent pas à garantir la robustesse.  
- Chaque cas limite doit avoir son test dédié.  
"""
```

```

import pytest

def calculer_moyenne(notes: list) -> float:
    if not notes:
        raise ValueError("Liste vide")
    return sum(notes) / len(notes)

# Cas nominal
def test_moyenne_normale():
    assert calculer_moyenne([10, 15, 20]) == pytest.approx(15.0)

# Cas limite : un seul élément
def test_moyenne_un_element():
    assert calculer_moyenne([12]) == pytest.approx(12.0)

# Cas limite : valeurs nulles
def test_moyenne_zeros():
    assert calculer_moyenne([0, 0, 0]) == pytest.approx(0.0)

# Cas limite : liste vide
def test_moyenne_liste_vide():
    with pytest.raises(ValueError, match="Liste vide"):
        calculer_moyenne([])

# Cas limite : valeurs négatives
def test_moyenne_negatifs():
    assert calculer_moyenne([-5, 5]) == pytest.approx(0.0)

```

18) Comment simuler une exception dans un mock ?

Énoncé

Comment tester le comportement du code quand une dépendance lève une exception ?

Solution

```

"""
Explication
- side_effect permet de définir l'exception levée par un mock.
- Permet de tester les branches d'erreur sans provoquer de vraie panne.
"""

```

```

import pytest
from unittest.mock import patch, MagicMock
from service import get_statut_serveur

def test_timeout_reseau():
    import requests

    with patch("service.requests.get") as mock_get:
        # Simuler un timeout réseau
        mock_get.side_effect = requests.exceptions.Timeout("Délai dépassé")

        # Le code doit gérer cette exception
        with pytest.raises(requests.exceptions.Timeout):
            get_statut_serveur("http://api.lente.com")

def test_erreur_connexion():
    import requests

    with patch("service.requests.get") as mock_get:
        mock_get.side_effect = requests.exceptions.ConnectionError("Hôte injoignable")

        with pytest.raises(requests.exceptions.ConnectionError):
            get_statut_serveur("http://api.hors-ligne.com")

```

19) Pourquoi éviter les dépendances entre tests ?

Énoncé

Quel problème survient quand des tests s'exécutent dans un ordre précis pour fonctionner ?

Solution

"""

Explication

- Les tests doivent être indépendants et idempotents.
- Un test qui dépend de l'état laissé par un test précédent est fragile.
- pytest peut exécuter les tests dans n'importe quel ordre (-p no:randomly).
- Solution : chaque test repart d'un état propre via les fixtures.

"""

```
import pytest
```

```

from panier import Panier

# Tests couplés — dangereux
panier_global = Panier() # État partagé entre tests

def test_ajouter_article_mauvais():
    panier_global.ajouter("Clavier", 49.99)
    assert len(panier_global.articles) == 1

def test_total_mauvais():
    # Dépend du test précédent pour avoir un article
    assert panier_global.total() > 0

# Tests indépendants — robustes
@pytest.fixture
def panier():
    return Panier() # Nouvel objet pour chaque test

def test_ajouter_article_correct(panier):
    panier.ajouter("Clavier", 49.99)
    assert len(panier.articles) == 1

def test_total_correct(panier):
    panier.ajouter("Souris", 29.99)
    assert panier.total() == pytest.approx(29.99)

```

20) Construire une suite de tests complète —cas réel

Énoncé

Proposer une suite de tests robuste pour un module métier complet.

Solution

```

"""
Explication
- Couvrir : cas nominal, cas limites, exceptions, interactions.
- Utiliser fixtures, parametrize, raises, approx.
- Viser un test lisible sans commentaires supplémentaires.
"""

import pytest
from compte import CompteBancaire

```

```

# — Fixtures —
@pytest.fixture
def compte_vide():
    return CompteBancaire("Alice", solde=0.0)

@pytest.fixture
def compte_approvisee():
    return CompteBancaire("Bechir", solde=1000.0)

# — Cas nominaux —
def test_solde_initial(compte_approvisee):
    assert compte_approvisee.solde() == pytest.approx(1000.0)

def test_depot_augmente_solde(compte_approvisee):
    compte_approvisee.deposer(500.0)
    assert compte_approvisee.solde() == pytest.approx(1500.0)

def test_retrait_diminue_solde(compte_approvisee):
    compte_approvisee.retirer(200.0)
    assert compte_approvisee.solde() == pytest.approx(800.0)

# — Cas limites —
def test_retrait_total(compte_approvisee):
    compte_approvisee.retirer(1000.0)
    assert compte_approvisee.solde() == pytest.approx(0.0)

def test_depot_sur_compte_vide(compte_vide):
    compte_vide.deposer(100.0)
    assert compte_vide.solde() == pytest.approx(100.0)

# — Exceptions —
def test_depot_montant_invalide(compte_vide):
    with pytest.raises(ValueError, match="Montant invalide"):
        compte_vide.deposer(0)

def test_retrait_solde_insuffisant(compte_approvisee):
    with pytest.raises(ValueError, match="Solde insuffisant"):
        compte_approvisee.retirer(9999.0)

# — Parametrize —
@pytest.mark.parametrize("depot, retrait, solde_final", [
    (500.0, 200.0, 1300.0),
    (100.0, 100.0, 1000.0),
    (0.01, 0.01, 1000.0),
])

```

```
def test_sequence_operations(compte_approvisionne, depot, retrait, solde_final):  
    compte_approvisionne.deposer(depot)  
    compte_approvisionne.retirer(retrait)  
    assert compte_approvisionne.solde() == pytest.approx(solde_final)
```